4.5.4 Choosing the right pattern

There is no one-size-fits-all architecture. At ACME Corp, centralized PDPs are employed for internal business applications where consistency and auditability are crucial. In contrast, distributed PDPs—embedded within authorization clients—power latency-sensitive services like ACMEFinance and ACMEProject, where local decision-making is essential. PEPs are strategically placed in application code, API gateways, or service mesh sidecars, depending on performance, visibility, and trust boundary requirements.

The goal isn't to rigidly follow a single model but rather to select the right balance of performance, reliability, observability, and control for each system. Whether centralized or distributed, embedded or sidecar-based, the core pattern remains the same: intercept the request, enrich it with context, evaluate it against policy, and enforce the decision.

Whether centralized or distributed, embedded or external, these patterns are shaped by a deeper principle: the separation of concerns in authorization architecture. Understanding why this separation exists is as crucial as understanding how the components fit together.

4.5.5 Why this separation matters

At first glance, separating enforcement, decision-making, policy management, and data sourcing may seem like overengineering. Wouldn't it be easier to simply check permissions directly in the app?

The problem is that's what most systems did for decades. And it worked—until it didn't.

As systems became more distributed, dynamic, and regulated, embedding access logic in application code or relying on hardcoded role checks became a liability. This approach led to brittle, fragmented systems where authorization rules were duplicated across services, making them difficult to update and nearly impossible to audit.